Code-Layout, Readability and Reusability

Date	22 June 2026
Team ID	
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code.	Yes	Adheres strictly to PEP 8 standards using 4 spaces for indentation across all files.
2	Proper File Structure	Files and folders are logically organized.	Yes	Separates presentation layouts, application logic, and data layer models safely.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Eliminates cryptic names; uses descriptive terms like `annual_rainfall` and `cloud_visibility`.
4	Function / Method Names	Functions are descriptively named	Yes	Uses clear action verbs for processes, such as `load_saved_artifacts()` and `generate_flood_prediction()`.
5	Code Comments	Inline and block comments explain logic.	Yes	Explains validation checkpoints, operational scaling steps, and model prediction trees.
6	Modular Design	Code is split into reusable functions/modules.	Yes	Decouples data processing workflows from the main backend web routing modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Centralizes scaling and serialization calls to avoid duplicate code in the application endpoints.
8	Error Handling	Exceptions and errors are handled gracefully.	Yes	Uses structured `try-except` blocks to handle missing model files or invalid web form values.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Data Preprocessing & Feature Scaling Utility	Python / Scikit-Learn	Reused during offline data training pipelines and within the active online Flask application layer.	High
2	Model Serialization Manager (`joblib` Helper)	Python / Joblib	Shared across the machine learning training script and the primary production backend router to load binary objects.	High
3	Form Validation Middleware	Python / Flask	Executed across all client form submission routes to sanitize text field inputs before running calculations.	Medium
4	Database Logging Utility	Python / SQLite (or CSV Writer)	Triggered by all analytical endpoints to save past operational run details directly into the tracking database.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Follows structural app layout guidelines cleanly. Project folders map perfectly to standard presentation, application, and storage tiers.
Readability	5	Adheres to explicit naming standards. Uses self-documenting code style, allowing external engineers to read and audit the weather parameter matrices easily.
Reusability	4	Preprocessing scaling tools and machine learning prediction tasks are packaged into dedicated functional blocks.
Documentation / Comments	5	Complete docstrings detail the parameters, expected matrix dimensions, and outputs of all core modules.
Overall Score	4.75 / 5	**Excellent.** A clean, professional codebase that runs seamlessly from collection scripts up to live cloud hosting runtimes.